Dynamic analyzer

Olimpia Dejko
Natalia Bardadyn

listopad 2026

Spis treści

1 Temat projektu **2**

2 Cel projektu **2**

3 Główne funkcjonalności **2**

4 Technologie **3**

5 Architektura systemu i przepływ danych **4**

6 Napotkane błędy i sposoby ich rozwiązania **5**

7 Bezpieczeństwo, walidacja i architektura proaktywna **6**

8 Plan realizacji **6**

9 Oczekiwany rezultat **7**

1 Temat projektu

Projekt polega na stworzeniu zaawansowanej aplikacji internetowej o charakterze badawczo-demonstracyjnym, umożliwiającej przeprowadzanie automatycznej hybrydowej (statycznej i dynamicznej) analizy plików wykonywalnych w formacie PE (*Portable Executable*) przeznaczonych dla systemu Microsoft Windows.
Aplikacja integruje inspekcję strukturalną kodu, wyliczanie kryptograficznych sum kontrolnych, badanie matematycznej entropii danych, klasyfikację wywołań funkcji systemowych API oraz próbne, kontrolowane uruchomienie procesu w środowisku izolowanym z limitowanym czasem wykonania.

2 Cel projektu

Głównym celem projektu jest opracowanie nowoczesnego, zintegrowanego narzędzia do analizy bezpieczeństwa oprogramowania, które pozwala na:

- Zapoznanie się z praktycznymi metodami statycznej oraz dynamicznej detekcji potencjalnie złośliwego oprogramowania.
- Zrozumienie wewnętrznej budowy plików binarnych systemu Windows (nagłówki PE, tablica importów, struktury sekcji).
- Zastosowanie metod matematycznych (entropia Shannona) do automatycznego wykrywania technik zaciemniania kodu (obfuskacji), pakowania oraz szyfrowania nagłówków.
- Automatyczne szacowanie poziomu zagrożenia (*Risk Scoring Engine*) na podstawie heurystyk bez konieczności ręcznej interpretacji surowych logów przez analityka.

3 Główne funkcjonalności

Planowane funkcjonalności aplikacji:

1. **Wybór programu testowego:** Interfejs webowy umożliwiający wskazanie pliku wykonywalnego za pomocą przeciągania (drag-and-drop) lub okna wyboru systemu.
2. **Uruchomienie analizy:** rozpoczęcie kontrolowanego wykonania wybranego programu.
3. **Obliczanie sum kontrolnych:** Automatyczne generowanie unikalnych kryptograficznych identyfikatorów pliku w standardzie MD5 oraz SHA-256.
4. **Monitorowanie plików:** wykrywanie wybranych operacji, takich jak tworzenie i modyfikowanie plików w obserwowanym katalogu.

5. **Inspekcja struktury PE (Portable Executable):** Odczyt nagłówków, liczby sekcji oraz daty kompilacji pliku za pomocą biblioteki `pefile`.
6. **Monitorowanie procesów:** zbieranie podstawowych informacji o analizowanym procesie oraz wybranych procesach potomnych.
7. **Monitorowanie sieci:** rejestrowanie kontrolowanych połączeń sieciowych wykonywanych przez program testowy.
8. **Analiza entropii Shannona:** Matematyczne badanie rozkładu bajtów w sekcjach pliku w zakresie od 0.0 do 8.0 w celu wykrycia upakowania, zaciemnienia lub szyfrowania kodu (próg alarmowy $H(X) > 7.2$).
9. **Kategoryzacja i detekcja niebezpiecznych API:** Skanowanie tabeli importów pod kątem obecności funkcji wysokiego ryzyka (np. iniekcje kodu takie jak `VirtualAlloc`, `CreateRemoteThread`, `LoadLibraryW`).
10. **Ocena ryzyka:** Algorytm sumujący punkty zagrożenia z analizy statycznej i klasyfikujący plik do odpowiedniego poziomu: *Niskie*, *Średnie* lub *Wysokie*.
11. **Kontrolowane uruchomienie z mechanizmem Timeout** - Uruchamianie procesu via subprocess.run z 3-sekundowym limitem czasowym zapobiegającym zawieszeniu serwera.
12. **Prezentacja wyników i wizualizacja:** Interaktywna tabela zdarzeń z podziałem na kategorie (*Pliki*, *Procesy*, *Sieć*, *Zasoby*) oraz nowoczesna, kolorowa karta oceny ryzyka w interfejsie.
13. **Generowanie raportu:** przygotowanie podsumowania przeprowadzonej analizy.

4 Technologie

Do realizacji projektu planujemy wykorzystać:

- **HTML** - strukturę interfejsu użytkownika.
- **CSS** - wygląd i układ aplikacji.
- **JavaScript** - obsługę interakcji użytkownika i aktualizację widoku.
- **Python** - implementację logiki analizatora, uruchamianie programów testowych i opracowywanie wyników.
  - **FastAPI** - nowoczesny, szybki framework webowy do budowy API.
  - **pefile** - biblioteka do parsowania i analizy struktur plików PE (Portable Executable), używana do odczytu nagłówków, sekcji i tabeli importów.
  - **Hashlib** - wbudowany moduł do obliczania kryptograficznych sum kontrolnych (MD5, SHA-256).

  - **UUID (Universally Unique Identifier)** - standard generowania unikalnych identyfikatorów używany do bezpiecznego przechowywania wgrywanych plików na serwerze.
  - **math** - wykorzystywany do obliczania entropii Shannona ($\log_2$).
  - **subprocess** - wbudowany moduł służący do kontrolowanego, dynamicznego uruchamiania procesów w odizolowanym środowisku.
- **Git i GitHub** - wersjonowanie kodu oraz współpracę nad projektem.

5 Architektura systemu i przepływ danych

Opis modelu klient-serwer, na którym opiera się aplikacja:

- **Backend (FastAPI/Python):** Odpowiada za odbieranie żądań HTTP, walidację strumienia plików, zarządzanie bezpiecznym katalogiem tymczasowym oraz parsowanie nagłówków PE za pomocą biblioteki pefile.
  1. Odbiór i walidacja pliku (rozszerzenie .exe, limit 150 MB).
  2. Zapis tymczasowy w folderze /uploads pod unikalną nazwą UUID.
  3. Analiza statyczna: obliczenie MD5 i SHA-256. Inspekcja struktury PE: odczyt nagłówka, daty kompilacji, wyliczenie entropii dla każdej sekcji, kategoryzacja importów DLL/API.
  4. Obliczenie punktacji ryzyka i przypisanie etykiety słownej.
  5. Analiza dynamiczna: wywołanie subprocess.run(timeout=3) i przechwycenie returncode.
  6. Bezpieczne usunięcie pliku tymczasowego (os.remove).
  7. Zwrócenie odpowiedzi JSON zawierającej listę zdarzeń (events) oraz poziom ryzyka (risklevel) do frontendu.
- **Frontend (HTML/CSS/JavaScript):** Odpowiada za interakcję z użytkownikiem, obsługę przeciągania plików (drag-and-drop), dynamiczne renderowanie tabeli zdarzeń, filtrowanie za pomocą zakładek oraz wizualizację postępu analizy.
- **Szczegółowy przepływ danych (Workflow):**
  1. Użytkownik przesyła plik `.exe` poprzez interfejs przeglądarkowy.
  2. Skrypt kliencki pakuje plik do obiektu `FormData` i przesyła go do endpointu `/api/analyze`.
  3. Serwer zapisuje plik tymczasowo w folderze `uploads/` pod losową nazwą UUID, eliminując ryzyko błędów ścieżek wynikających ze znaków specjalnych.
  4. Moduł oblicza hashe MD5 i SHA-256 oraz parsije nagłówek PE, wyznaczając entropię sekcji oraz analizując tablicę importów DLL.

5. Algorytm oceny ryzyka wyznacza punktację (*risk_score*) i przypisuje poziom zagrożenia.
6. Uruchamiany jest proces testowy w trybie kontrolowanym (`subprocess.run`), a uzyskany kod powrotu i zużycie zasobów (RAM/CPU) trafiają do rejestru zdarzeń.
7. Plik tymczasowy jest automatycznie usuwany z dysku serwera (`os.remove`), a wynik w formacie JSON jest zwracany do frontendu w celu natychmiastowej wizualizacji.

6 Napotkane błędy i sposoby ich rozwiązania

Podczas implementacji kolejnych funkcji projektu napotkano na kilka wyzwań związanych ze stanem interfejsu (UI) oraz synchronizacją kodu. Poniżej zebrano najważniejsze z nich wraz z zastosowanymi poprawkami:

1. **Przeciążenie serwera przez pliki o niestandardowym rozmiarze**
   Opis: Przesłanie skrajnie dużego pliku mogłoby doprowadzić do wyczerpania pamięci RAM serwera lub nadmiernego obciążenia procesora podczas parsowania struktury PE.
   Rozwiązanie: Wprowadzono restrykcyjną walidację rozmiaru przesyłanego strumienia danych po stronie backendu (maksymalnie 150 MB), odrzucającą nieodpowiednie pliki na samym początku żądania HTTP.

2. **Znikanie przycisku wyboru pliku po błędnej walidacji lub niepowodzeniu analizy**
   Opis: Gdy użytkownik przesłał plik niebędący plikiem .exe lub gdy wystąpił błąd komunikacji z serwerem, kluczowe elementy interfejsu (takie jak przycisk wyboru/zmiany pliku) znikały, pozostawiając interfejs w zablokowanym stanie.
   Rozwiązanie: Zmodyfikowano logikę w pliku app.js, rozdzielając przycisk wyboru pliku od stanu analizy. Zapewniono, że przycisk wyboru pliku jest zawsze widoczny, niezależnie od wystąpienia błędu czy statusu przesłanego pliku.

3. **Znaki specjalne w nazwach plików**
   Opis: Użytkownicy mogli przesyłać pliki zawierające spacje, polskie znaki diakrytyczne lub znaki specjalne, co na backendzie mogłoby prowadzić do błędów ścieżek lub podatności bezpieczeństwa.
   Rozwiązanie: Zastosowano generowanie losowych, unikalnych nazw tymczasowych przy użyciu standardu UUID dla każdego wgrywanego pliku. Dzięki temu operacje plikowe na serwerze są przewidywalne i bezpieczne.

4. **Surowe powiadomienia systemowe zaburzające estetykę**
   Opis: Komunikaty o błędach (np. nieprawidłowe rozszerzenie pliku lub funkcja w budowie) były obsługiwane przez standardowe okienka alert(), co psuło spójność ciemnego, nowoczesnego motywu aplikacji.

Rozwiązanie: Zaimplementowano dedykowany komponent baneru powiadomień stylizowany w CSS na wzór powiadomień typu toast. Dodano węższą, wyśrodkowaną formę oraz płynne animacje pojawiania się i wygaszania, dzięki czemu błędy znikają samoczynnie po kilku sekundach lub po kliknięciu krzyżyka.

7 Bezpieczeństwo, walidacja i architektura proaktywna

Opis mechanizmów chroniących aplikację i serwer przed niepożądanym zachowaniem:

- **Walidacja rozszerzeń i typów:** Restrykcyjne sprawdzenie, czy przesyłany plik posiada rozszerzenie .exe zarówno po stronie klienta, jak i serwera.
- **Ochrona przed Path Traversal i kolizjami nazw:** Zastosowanie losowych identyfikatorów UUID do nadawania nazw plikom w folderze roboczym serwera, co uniezależnia system od znaków specjalnych i spacji w oryginalnej nazwie pliku.
- **Kontrola rozmiaru danych:** Wprowadzenie twardego limitu rozmiaru pliku (maksymalnie 150 MB), zapobiegającego wyczerpaniu pamięci RAM serwera.

8 Plan realizacji

Harmonogram prac nad projektem obejmuje następujące etapy:

1. **Projekt interfejsu i warstwy wizualnej:** Przygotowanie struktury HTML oraz stylów CSS dla motywu ciemnego, uwzględniając responsywność i obsługę zdarzeń przeciągania plików (drag-and-drop).
2. **Implementacja rdzenia backendu:** Uruchomienie serwera opartego o framework FastAPI w języku Python oraz konfiguracja bezpiecznego katalogu tymczasowego z obsługą identyfikatorów UUID.
3. **Moduł analizy statycznej PE:** Integracja biblioteki `pefile` w celu odczytu nagłówków, sekcji, daty kompilacji oraz tabeli importów.
4. **Zaawansowana detekcja statyczna:** Implementacja obliczania entropii Shannona dla sekcji pliku ($H(X) > 7.2$) oraz wykrywania podejrzanych funkcji API (iniekcje pamięci, dynamiczne ładowanie).
5. **Moduł analizy dynamicznej i piaskownicy:** Wdrożenie kontrolowanego uruchamiania procesu za pomocą modułu `subprocess` z limitem czasowym (*timeout*) zapobiegającym blokowaniu serwera.
6. **Silnik oceny ryzyka i powiadomienia UI:** Stworzenie algorytmu wyznaczającego poziom zagrożenia (*Niskie*, *Średnie*, *Wysokie*) oraz zaprojektowanie nowoczesnej karty ryzyka i powiadomień typu *toast*.

7. **Testy, dokumentacja i finalizacja:** Przeprowadzenie testów integracyjnych na plikach wykonywalnych, przygotowanie pełnej dokumentacji technicznej w formacie LaTeX oraz finalizacja aplikacji.

9 Oczekiwany rezultat

Rezultatem projektu ma być działająca aplikacja z interfejsem graficznym, która umożliwi uruchamianie przygotowanych programów testowych, rejestrowanie wybranych zdarzeń podczas ich działania oraz prezentowanie wyników w postaci czytelnego raportu. Projekt ma również umożliwić praktyczne zaprezentowanie podstaw dynamicznej analizy oprogramowania i sposobów wykorzystania informacji o zachowaniu programu w kontekście cyberbezpieczeństwa.